

Linguagem de expressão 1

- Expressões constantes
- BNF
- Exemplos de programas
- Diagrama de classes
- Implementações
- Labs e subprojeto

Expressões constantes

- A Linguagem de Expressão 1 (le1) é composta essencialmente por expressões.
- Nesta linguagem, um **programa** é uma expressão e,
- assim, sua execução (avaliação) produz um **valor** como resultado.

Expressões constantes

- Um valor é algo que pode ser avaliado, armazenado, incorporado em estruturas de dados, passado como parâmetro, retornado como resultado, ...
- Ou seja, um valor é uma entidade que existe durante uma computação

Expressões constantes

- **Tipos** são conjuntos de valores e operações associadas
- Valores de um tipo devem exibir comportamento uniforme em relação às **operações** sobre o tipo

BNF

- A le1 envolve conceitos de tipo, valores constantes e operadores sobre expressões.

Programa ::= Expressão

Expressao ::= Valor | ExpUnaria | ExpBinaria

Valor ::= ValorConcreto

ValorConcreto ::= ValorInteiro | ValorBooleano | ValorString

ExpUnaria ::= “-” Expressao | “not” Expressao | “length” Expressao

ExpBinaria ::= Expressao “+” Expressao | Expressao “-” Expressao |
Expressao “and” Expressao | Expressao “or” Expressao |
Expressao “==” Expressao | Expression “++” Expression

Figure 8. BNF para a Linguagem de Expressão 1

BNF

- De acordo com a BNF da linguagem, um programa é uma expressão que pode ser um valor, uma expressão unária ou binária.
- Na le1, consideramos apenas valores concretos como inteiros, booleanos e string (que admitem comparação sintática por igualdade).

BNF

- Como exemplos de expressão unária, consideramos:
- a troca de sinais de inteiros,
- a negação de um booleano
- e a operação que retorna o tamanho de um string.

BNF

- As expressões binárias envolvem:
- soma e subtração sobre inteiros,
- conjunção e disjunção lógica,
- igualdade sobre todos os tipos de expressão
- e concatenação de strings.

Exemplos de programas

- Exemplos de programas válidos nesta linguagem envolvem apenas constantes,
- `length("abc") + 3`
- `true and false`
- `"curso" ++ " de " ++ " paradigmas"`
- quando executados, retornam 6, false e "curso de paradigmas", como esperado.
- Expressões como `1 + true` resultam em erro de tipo.

Diagrama de classes

- A figura a seguir mostra estrutura do projeto desta linguagem, através de um diagrama de classes UML.
- No projeto orientado a objetos das linguagens, cada item da BNF é uma classe, interface ou classe abstrata de Java.
- Classes adicionais são introduzidas para modelar estruturas internas do interpretador, como o contexto.

Diagrama de classes

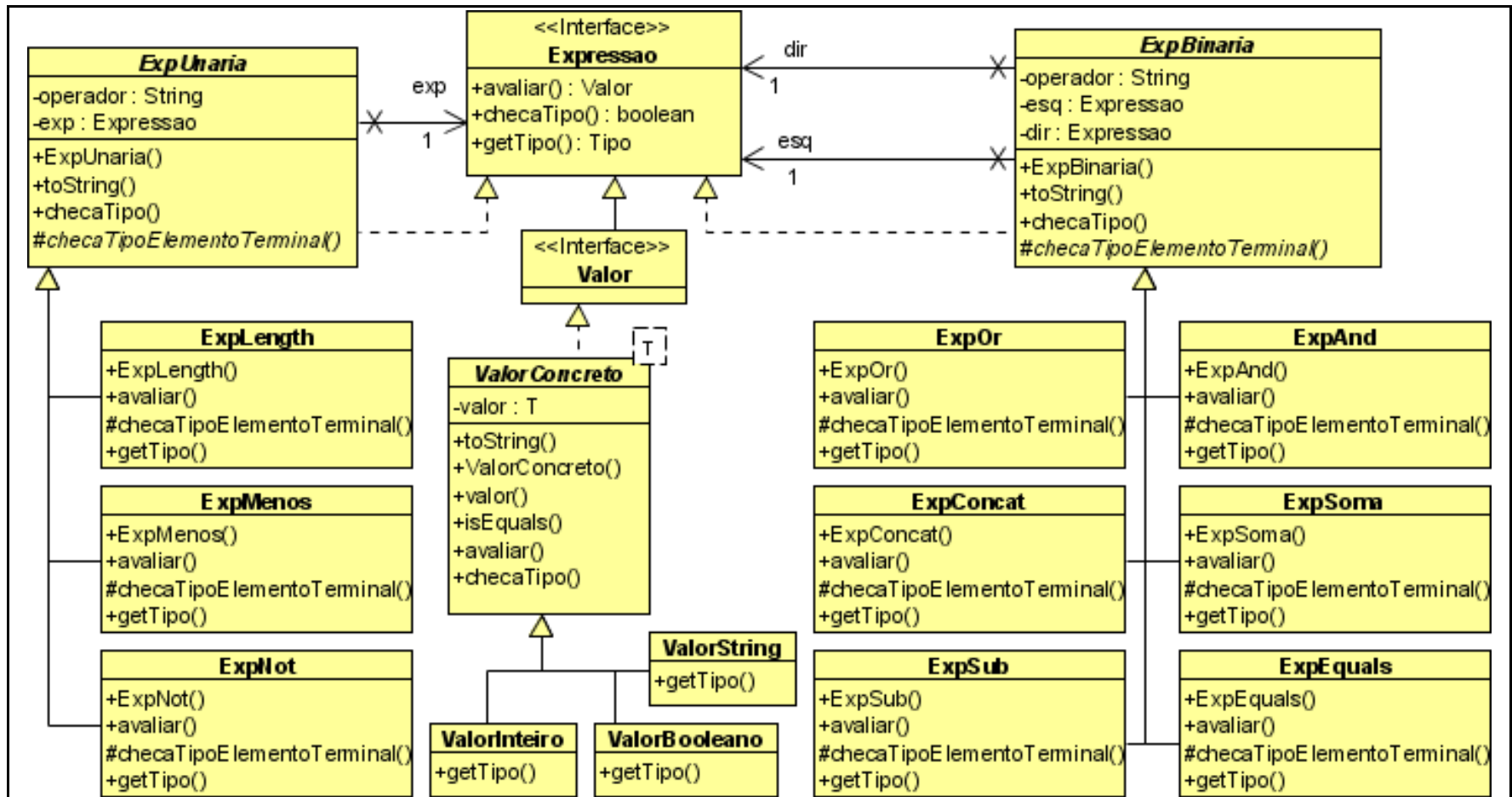


Figura 10 Diagrama de classes para a Linguagem de Expressão 1

Diagrama de classes

- A hierarquia da BNF é modelada através de herança e subtipos (extends e implements).
- Os elementos não terminais são usualmente mapeados em interfaces ou em classes abstratas.

Diagrama de classes

- Por exemplo, o item Expressao é modelado como uma interface.
- A razão é que não há qualquer elemento comum entre os diversos tipos de expressão que possam ser compartilhados, a não ser a assinatura dos métodos;

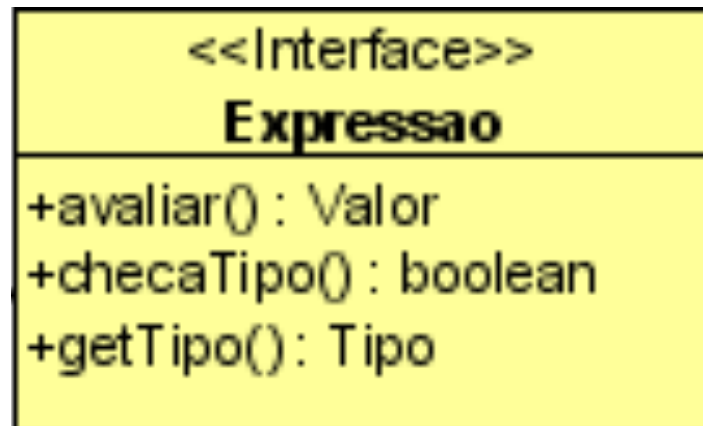


Diagrama de classes

- Já ExpressaoUnaria e ExpressaoBinaria são modeladas como classes abstratas.
- no caso de expressões unárias todas possuem um operador e um argumento (que também é uma expressão),
- de forma que há a possibilidade de um compartilhamento estrutural (atributos);

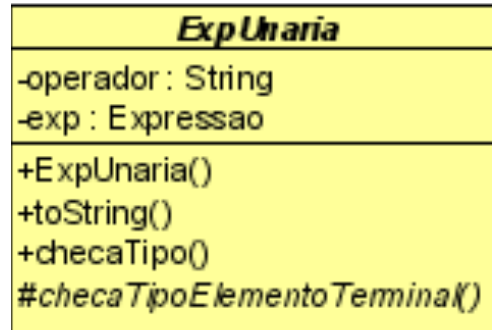


Diagrama de classes

- o mesmo ocorre no caso de expressões binárias,
- que possuem 2 argumentos (expressões) e um operador.

<i>ExpBinaria</i>
-operador : String -esq : Expressao -dir : Expressao
+ExpBinaria() +toString() +checaTipo() #checaTipoElementoTerminal()

Diagrama de classes

- O elemento não terminal Valor também é claramente capturado por uma interface, já que valores concretos e abstratos não compartilham estrutura.
- Em alguns casos, parametrização de classes fornece poder de expressão adicional.



Diagrama de classes

- é possível representar ValorConcreto como uma classe abstrata parametrizada (usando o conceito de generics) com o tipo do atributo encapsulado;
- elementos terminais como ValorInteiro, ValorBooleano e ValorString são instâncias desta classe parametrizada abstrata.

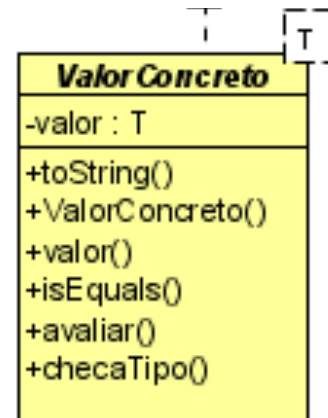
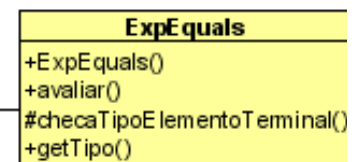
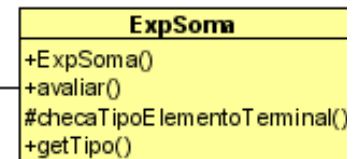
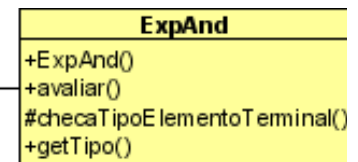
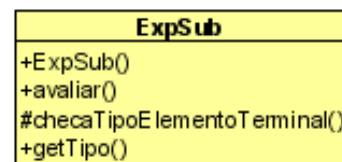
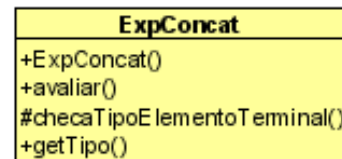
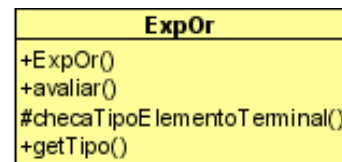
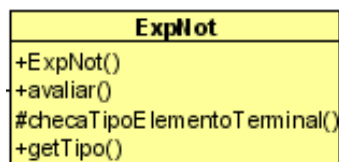
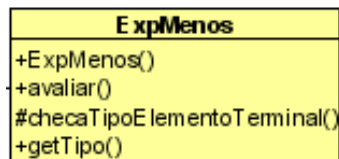
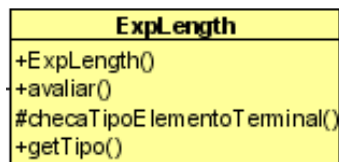


Diagrama de classes

- Os elementos da BNF que implementam cada um dos operadores unários e binários também são implementados por classes concretas.



Implementações

- Com relação ao código, a interface `Expressao` inclui as assinaturas dos métodos `avaliar`, `checaTipo` e `getTipo`.
- O primeiro retorna o valor da expressão avaliada.
- O método `checaTipo` retorna um valor booleano indicando se a expressão é ou não bem tipada.
- O último método retorna o tipo da expressão.

Implementações

- public interface Expressao {
 - Valor avaliar();
 - boolean checaTipo();
 - Tipo getTipo();
- }

Implementações

- A classe que implementa um Programa contém um atributo (a expressão a ser avaliada) e
- invoca estes métodos para verificar se a expressão é bem tipada e,
- caso seja, avaliá-la e retornar o resultado produzido

Implementações

- `public class Programa{`
- `private Expressao exp;`
- `public Valor executar() {`
- `Valor result = exp.avaliar();`
- `System.out.println(result);`
- `return result;`
- `}`
- `public boolean checaTipo() {`
- `return exp.checaTipo();`
- `}`

Implementações

- //classe Programa (continuação)
- public Programa(Expressao exp){
- this.exp = exp;
- }
- public Expressao getExpressao() {
- return exp;
- }
- }

Implementações

- A interface Valor estende Expressao mas não adiciona qualquer assinatura de método.
- `public interface Valor extends Expressao{`
- `}`

Implementações

- Já ValorConcreto é implementado por uma classe parametrizada (genérica)
- A classe é parametrizada pelo tipo T,
- que pode então ser instanciado com classes representando inteiros, booleanos e string, para criar valores dos respectivos tipos
- como ilustrado pelo classe ValorInteiro, na figura a seguir.

Implementações

```
public abstract class ValorConcreto<T> implements Valor {
    private T valor;
    public T valor() {return valor;}
    public boolean isEquals(ValorConcreto<T> obj) {return valor().equals(obj.valor()); }
    public Valor avaliar() {return this;}
    public boolean checaTipo() {return true;}
}

public class ValorInteiro extends ValorConcreto<Integer> {
    public Tipo getTipo() {return Tipo.TIPO_INTEIRO;}
}
```

Figura 11. Código das classes ValorConcreto e ValorInteiro

Implementações

- O atributo valor representa o valor encapsulado, que pode ser consultado usando o método valor().
- O método isEqual compara dois valores e implementa o operador ==.
- A avaliação de um valor retorna o próprio e um valor está sempre bem tipado.

Implementações

- Construtores e alguns métodos responsáveis pela impressão de um valor são omitidos no código apresentado.
- A classe ValorInteiro herda e instancia a classe ValorConcreto, adicionando a implementação do método que retorna o tipo específico da constante.
- O código de ValorBooleano e ValorString é análogo.

Implementações

- Como um exemplo de uma expressão, vamos apresentar o código da classe ExpSoma,
- que implementa a operação binária de soma de duas expressões inteiras.
- A avaliação de uma expressão binária envolve a avaliação de cada uma das subexpressões.

Implementações

```
public class ExpSoma extends ExpBinaria {  
    public Valor avaliar() {  
        return new ValorInteiro(  
            ((ValorInteiro) getEsq()).avaliar().valor() + ((ValorInteiro) getDir()).avaliar().valor() );  
    }  
    public boolean checaTipo() {  
        return (getEsq().getTipo().eInteiro() && getDir().getTipo().eInteiro());  
    }  
    public Tipo getTipo() {return Tipo.TIPO_INTEIRO;}  
}
```

Figura 12. Código da classe ExpSoma

Implementações

- Os valores parciais são somados e o resultado final é encapsulado como um objeto do tipo `ValorInteiro`.
- Tal expressão é bem tipada se e somente se ambos os argumentos forem de fato do tipo inteiro.
- O tipo da expressão como um todo é, obviamente, do tipo inteiro.

Atividade 1

- Usando a le1, elabore uma classe em Java chamada de **Exemplo1** para avaliar a seguinte expressão:
- $4 + 12 - 3$

Atividade 2

- Usando a le1, elabore uma classe em Java chamada de **Exemplos** para avaliar as seguintes expressões:
- $-4 + 12 - 3$
- `length("abc") + 3`
- `true and false`
- `"curso" ++ " de " ++ " paradigmas"`
- `1 + true`

Subprojeto le1

- Converter toda a linguagem le1 que está escrita em Java para Python (inclusive com a classe Exemplos).
- Esse subprojeto completo deve ser entregue no final do curso.

Leitura recomendável

- Conceitos e Paradigmas de Programação via Projeto de Interpretadores
- Augusto Sampaio e Antônio Maranhão
- Material em preparação para o JAI-2008 (Jornadas de atualização em Informática – Sociedade Brasileira de Computação)